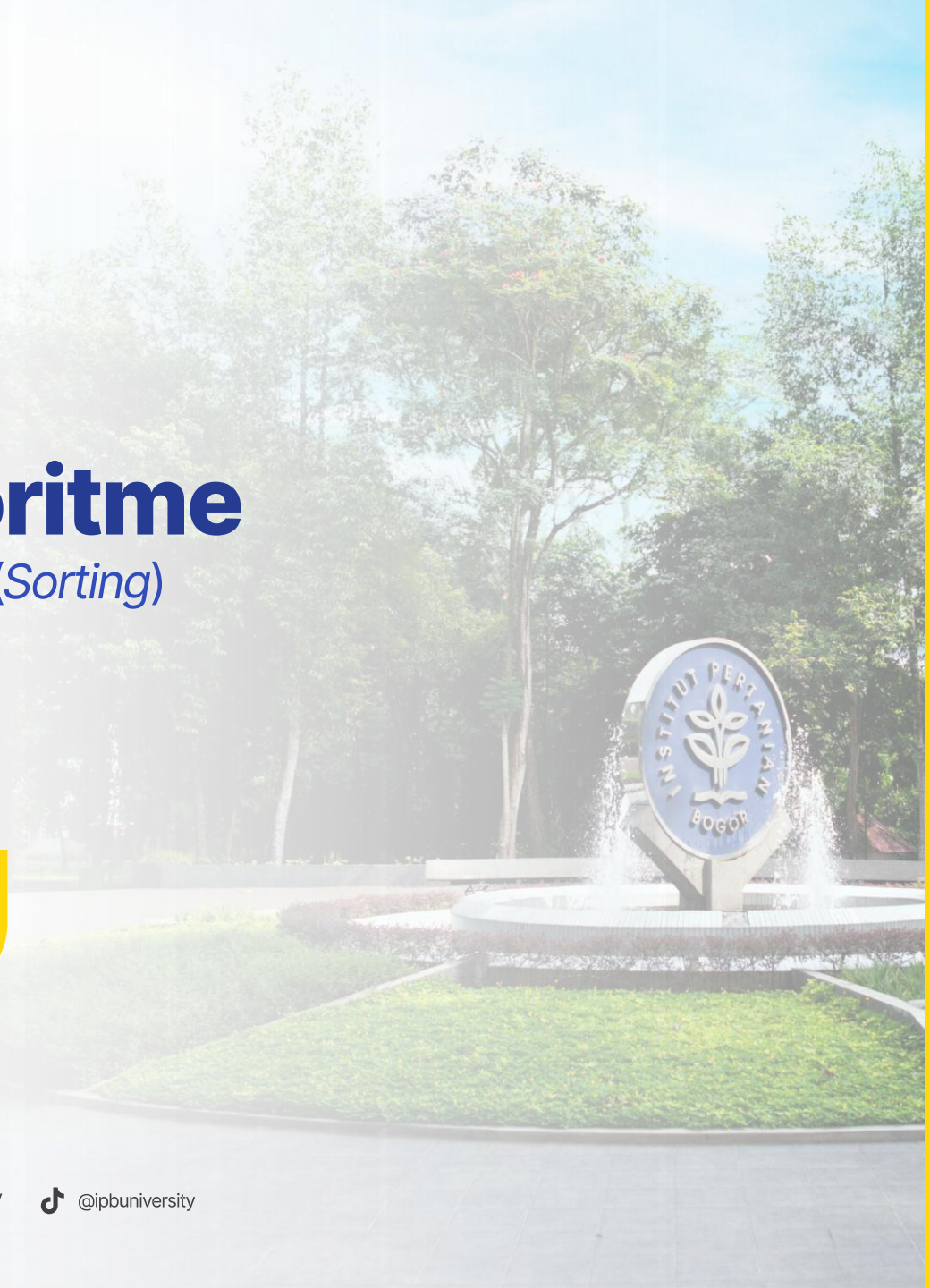
IPB University
Bogor Indonesia

DIKTISAINTEK BERDAMPAK

MK Analisis Algoritme

Pertemuan 7: Algoritme Pengurutan (*Sorting*)



Tim Pengajar
MK Analisis Algoritme

Tujuan Pembelajaran

- Mahasiswa mampu menjelaskan beberapa jenis algoritme pengurutan (sorting) yang umum digunakan
- Mahasiswa mampu menganalisis kompleksitas dari algoritme-algoritme pengurutan tersebut
- Mahasiswa mampu menjelaskan keunggulan dan kelemahan dari masing-masing algoritme pengurutan
- Mahasiswa mampu menjelaskan bagaimana analisis kompleksitas digunakan dalam dunia nyata (industri)

Review Algoritme Pengurutan

- Algoritme pencarian:

diberikan sebuah array $A[1 \ldots n]$, susun ulanglah array tersebut sedemikian rupa sehingga semua elemen-elemen A terurut (naik/turun)

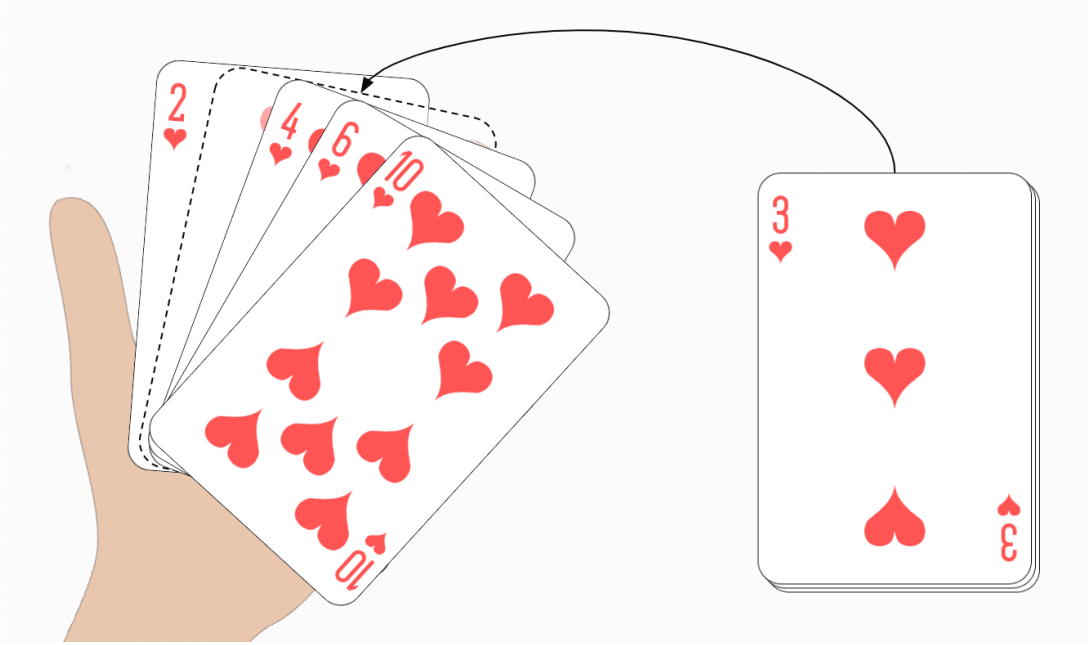
Ada beberapa buah algoritme yang paling sering dipelajar:

- Bubble Sort
- Insertion Sort
- Merge Sort
- Quick Sort

IPB University
Bogor Indonesia

Insertion Sort

- Ide utama: **sisipkan** elemen-elemen yang belum terurut ke posisi yang **seharusnya**
- Seperti biasa dilakukan ketika mengurutkan kartu-kartu
- Posisi yang benar dicari dengan membandingkan elemen yang disisipkan dengan nilai-nilai berikutnya



IPB University
Bogor Indonesia

Insertion Sort

- https://youtu.be/Q1JdRUh1_98

```
1  INSERTION-SORT(A[1..n]):
2      for i ← 2 to n:
3          key ← A[i]
4          j ← i - 1
5          while j ≥ 1 and A[j] > key:
6              A[j+1] ← A[j]
7              j ← j - 1
8          A[j+1] ← key
9
```

IPB University
Bogor Indonesia

Insertion Sort

- Best case: **O(n)**
  - Kapan?
- Worst case: $\boldsymbol{O(N^2)}$
- Average case: $\boldsymbol{O(N^2)}$
  - **Bagaimana membuktikan?**

IPB University
Bogor Indonesia

Kondisi Best Case

```
1  INSERTION-SORT(A[1..n]):
2      for i ← 2 to n:
3          key ← A[i]
4          j ← i - 1
5          while j ≥ 1 and A[j] > key:
6              A[j+1] ← A[j]
7              j ← j - 1
8          A[j+1] ← key
9
```

- **Array sudah terurut menaik (sorted ascending).**
- Contoh: [1, 2, 3, 4, 5] -→ [2, 3, 4, 1, 5]
- [(2, mutia) (3,faqih) (2, insan) (3, gilang) (1, daffa)] →
- [(1,daffa) (2,mutia) (2, insan) (3,faqih) (3, gilang)] → stabilitas
- Pada kondisi ini:
- Loop luar tetap berjalan dari i = 2 hingga N (≈ N langkah).
- Tetapi **inner loop (shifting)** tidak melakukan pergeseran, karena setiap elemen yang baru akan dibandingkan sekali dan langsung berhenti (karena syarat key < A[j] tidak pernah benar).

IPB University
Bogor Indonesia

Kondisi Worst Case

- **Kapan terjadi?**
  - Saat array terurut **menurun**.
  - Setiap elemen yang dimasukkan harus dibandingkan dan digeser dengan semua elemen sebelumnya.
  - Jumlah perbandingan:
- $1 + 2 + 3 + \cdots + (N - 1) = \frac{N(N-1)}{2}$
  - Jadi **O(N²)**.

IPB University
Bogor Indonesia

Kondisi Average Case

- Saat kita menyisipkan elemen ke-i (anggap A[i]), kita harus mencari posisi yang tepat di antara elemen sebelumnya (A[1..i-1]).
- Jika array acak, peluang A[i] lebih kecil dari sebagian elemen sebelumnya adalah **sama besar** untuk setiap posisi.
- Artinya, lokasi penyisipan bisa jatuh di mana saja dengan **probabilitas yang hampir seragam**.
- Contoh untuk 5 elemen [? ? ? ? ?]:
- Elemen ke-5 bisa jadi paling kecil → harus geser semua 4 sebelumnya.
- Bisa jadi paling besar → tidak geser sama sekali.
- Bisa jatuh di tengah → geser sebagian.
  Secara rata-rata, **setengah dari 4 elemen** harus digeser.

$$\frac{1}{2}(1 + 2 + \cdots + (N-1)) \approx \frac{N^2}{4}$$

Tetap dalam orde $\mathbf{O(N^2)}$.

IPB University
Bogor Indonesia

Merge Sort

- Prinsip utama: **Divide and Conquer**
- Secara **rekursif**, bagi array menjadi 2 (kiri dan kanan)
  - Lakukan terus sampai mendapatkan elemen berukuran 1
- Lakukan **merge:**
  - Gabungkan dua array yang sudah terurut menjadi satu array terurut
- Kunci komputasi ada di **merge. Mengapa?**

IPB University
Bogor Indonesia

Merge Sort

```
1  MERGE-SORT(A[1..n]):
2      if n ≤ 1: return
3      mid ← ⌊n/2⌋
4      // sort left half A[1..mid] and right half A[mid+1..n]
5      MERGE-SORT(A[1..mid])
6      MERGE-SORT(A[mid+1..n])
7      MERGE(A, 1, mid, n)
```

IPB University
Bogor Indonesia

Divide (membagi array) itu murah

- Proses "membagi array jadi 2" hanya butuh operasi indeks dan pointer.
- Tidak ada banyak perbandingan atau perpindahan data.
- Kompleksitas pembagian ≈ **O(1)** tiap level.

IPB University
Bogor Indonesia

Conquer (rekursi) hanya mengulang pembagian

- Saat rekursi berjalan, array terus dipecah jadi lebih kecil.
- Sampai level dasar (subarray ukuran 1), tidak ada operasi berarti.
- Biaya utama **bukan di sini**, karena ini hanya memanggil fungsi rekursif.

IPB University
Bogor Indonesia

Merge Sort

```
1  MERGE(A, ℓ, m, r):                    // merges A[ℓ..m] and A[m+1..r] (both sorted)
2      nL ← m - ℓ + 1
3      nR ← r - m
4      L ← new array[1..nL]
5      R ← new array[1..nR]
6      for i ← 1..nL: L[i] ← A[ℓ + i - 1]
7      for j ← 1..nR: R[j] ← A[m + j]
8
9      i ← 1; j ← 1; k ← ℓ
10     while i ≤ nL and j ≤ nR:
11         if L[i] ≤ R[j]:                 // ≤ keeps stability
12             A[k] ← L[i];  i ← i + 1
13         else:
14             A[k] ← R[j];  j ← j + 1
15         k ← k + 1
16
17     while i ≤ nL: A[k] ← L[i]; i ← i + 1; k ← k + 1
18     while j ≤ nR: A[k] ← R[j]; j ← j + 1; k ← k + 1
```

IPB University
Bogor Indonesia

Merge = operasi mahal

- Untuk menggabungkan dua subarray terurut, kita harus:
  - Membandingkan elemen dari dua subarray (left dan right).
  - Memilih elemen lebih kecil untuk ditempatkan ke array hasil.
  - Dilakukan berulang sampai semua elemen habis.
- Jika panjang array = n, maka merge butuh **O(n)** per langkah.
- **Biaya total ditentukan oleh merge**
- Tiap level rekursi ada proses merge dengan total n per level.
- Jumlah level = log n (karena selalu dibagi dua).
- Jadi total biaya:

$$T(n) = O(n \log n)$$

IPB University
Bogor Indonesia

Merge Sort

- Relasi rekursi

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

2T(n/2): karena setiap kali array dibagi dua, kita memanggil rekursi untuk bagian kiri dan kanan.
O(n): karena untuk menggabungkan (merge) dua bagian terurut butuh waktu linear terhadap ukuran array.

- Menggunakan Master Theorem: $O(n) = n \log n$
- Apakah worst-case dan best-case sama?

IPB University
Bogor Indonesia

Ya, kompleksitas merge-sort sama untuk semua kondisi

Struktur pembagian selalu sama

Merge Sort **selalu** membagi array menjadi dua bagian hingga ukuran 1.

Mau arraynya sudah terurut, terbalik, atau acak → proses pembagian tidak berubah.

Jadi, bagian "divide" tidak pernah lebih cepat.

Proses merge selalu O(n)

Pada setiap level, Merge Sort **harus membandingkan hampir semua elemen** dari dua subarray terurut untuk digabungkan.

Tidak ada "shortcut" seperti di Insertion Sort (yang bisa cepat kalau data sudah sorted).

Jadi, setiap merge tetap butuh **O(n)**.

Pohon rekursi tetap log n

Jumlah level selalu **$\log_2(n)$** karena array dibagi dua terus.

Biaya tiap level tetap **O(n)**.

Total = **O(n log n)** dalam semua kondisi.

IPB University
Bogor Indonesia

Quick Sort

- Prinsip: mirip seperti merge sort, kita menerapkan prinsip **divide and conquer**
- Tidak membagi langsung array, namun melakukan pemilihan elemen **pivot**
- Array dibagi menjadi dua bagian:
  - Kiri: semua elemen ≤ **pivot**
  - Kanan: semua elemen ≥ **pivot**
- Lakukan secara rekursif sampai mendapatkan array berukuran 1/0

IPB University
Bogor Indonesia

Quick Sort

```
1  QUICKSORT(A, ℓ, r):
2      if ℓ ≥ r: return
3      p ← PARTITION(A, ℓ, r)
4      QUICKSORT(A, ℓ, p)
5      QUICKSORT(A, p+1, r)
6
7  PARTITION(A, ℓ, r):              // returns index p such that
8      pivot ← A[ ⌊(ℓ + r)/2⌋ ]    // all in [ℓ..p] ≤ pivot ≤ all in [p+1..r]
9      i ← ℓ
10     j ← r
11     while true:
12         while A[i] < pivot: i ← i + 1
13         while A[j] > pivot: j ← j - 1
14         if i ≥ j: return j
15         swap A[i], A[j]
16         i ← i + 1
17         j ← j - 1
```

IPB University
Bogor Indonesia

Quick Sort

- Misalkan kita memilih pivot elemen ke-i
  - Ukuran sisi kiri = i
  - Ukuran sisi kanan = n – i – 1
  - Biaya partisi = cn = O(n)
- Relasi rekursi

$$T(n) = T(i) + T(n - i - 1) + cn$$

IPB University
Bogor Indonesia

Quick Sort

- Kasus terburuk ketika **tidak seimbang (i = 0 atau n-1)**

$$T(n) = T(n-1) + cn \Rightarrow T(n) = \Theta(n^2)$$

- Kasus terbaik ketika selalu seimbang (pivot berada di tengah):

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \Rightarrow T(n) = \Theta(n \log n)$$

IPB University
Bogor Indonesia

Quick Sort

- Kasus rata-rata dapat diturunkan dengan menggunakan analisis nilai ekspektasi:

$$\mathbb{E}[T(n)] = \frac{1}{n}\sum_{i=0}^{n-1}\left(\mathbb{E}[T(i)] + \mathbb{E}[T(n-1-i)]\right) + cn = \frac{2}{n}\sum_{k=0}^{n-1}\mathbb{E}[T(k)] + cn.$$

- **1/n** → Karena pivot bisa jatuh di posisi manapun dari 0 sampai n-1 dengan peluang sama besar.
- $E[T(i)]$ →biaya quicksort untuk bagian kiri (ukuran i).
- $E[T(n-1-i)]$ →biaya quicksort untuk bagian kanan (ukuran n-1-i).
- $+cn$→ biaya partisi (sekali scanning array sepanjang n).

- Yang menghasilkan

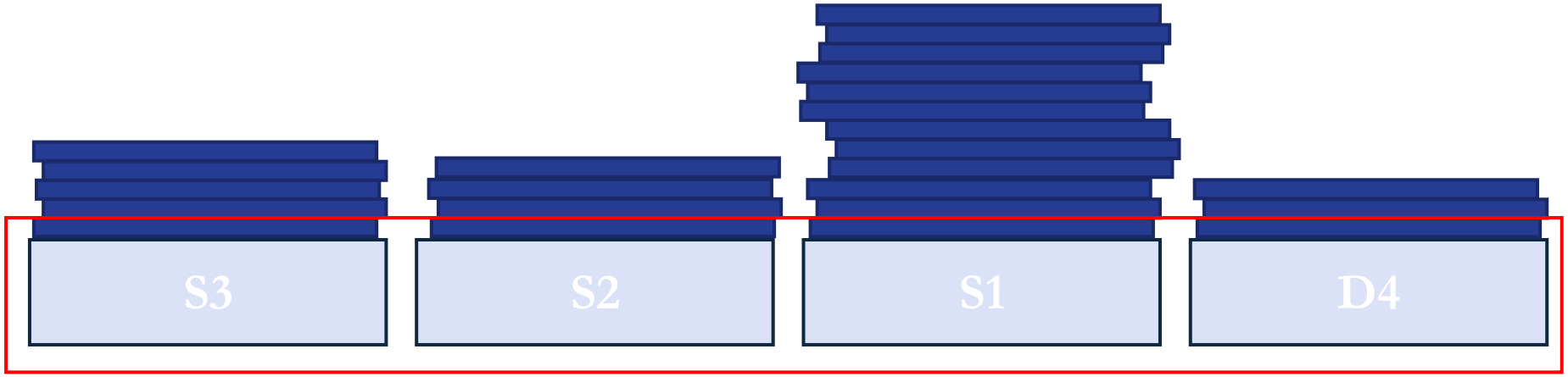
$$\mathbb{E}[T(n)] = \Theta(n \log n).$$

Counting Sort

- Counting sort adalah teknik pengurutan berdasarkan kunci dalam rentang tertentu.
- Metode ini bekerja dengan menghitung jumlah objek yang memiliki nilai kunci yang berbeda
- Kemudian melakukan beberapa perhitungan aritmatika untuk menentukan posisi setiap objek dalam urutan.
- Cek ini: https://www.youtube.com/watch?v=0B33As8jPgo

https://www.geeksforgeeks.org/counting-sort/

Counting Sort



Counting Sort memerlukan wadah (*bin*). Latihan #1 dan Latihan #2 keduanya melibatkan wadah di dalam problem.

Counting Sort cepat, namun hanya bisa bekerja pada problem dengan *constraint* tertentu.

Counting Sort

```c
int main() {
    int array[] = {2, 5, 3, 0, 2, 3, 0, 3};
    int size = sizeof(array) / sizeof(array[0]);

    printf("Array sebelum diurutkan: \n");
    for (int i = 0; i < size; i++) {
        printf("%d ", array[i]);
    }
    printf("\n");

    // Panggil fungsi Counting Sort
    countingSort(array, size);

    printf("Array setelah diurutkan: \n");
    for (int i = 0; i < size; i++) {
        printf("%d ", array[i]);
    }
    printf("\n");

    return 0;
}
```

Input :

```
Array sebelum diurutkan:
2 5 3 0 2 3 0 3
```

Output :

```
Array setelah diurutkan:
0 0 2 2 3 3 3 5
```

Counting Sort

```
void countingSort(int array[], int size) {
    // Temukan elemen terbesar dalam array untuk menentukan ukuran countArray
    int max = array[0];
    for (int i = 1; i < size; i++) {
        if (array[i] > max) {
            max = array[i];
        }
    }

    // Buat countArray dan inisialisasi dengan 0
    int countArray[max + 1];
    memset(countArray, 0, sizeof(countArray));
```

2 5 3 0 2 3 0 3

max = 5

countArray

Counting Sort

```
// Hitung frekuensi kemunculan setiap elemen dalam inputArray
for (int i = 0; i < size; i++) {
    countArray[array[i]]++;
}

// Mengubah countArray menjadi cumulative countArray
for (int i = 1; i <= max; i++) {
    countArray[i] += countArray[i - 1];
}
```

array	2	5	3	0	2	3	0	3

countArray	2	0	2	3	0	1

countArray	2	2	4	7	7	8

Counting Sort

```
    // Buat outputArray
    int outputArray[size];

    // Isi outputArray dengan elemen-elemen yang terurut
    for (int i = size - 1; i >= 0; i--) {
        outputArray[countArray[array[i]] - 1] = array[i];
        countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
    }

    // Salin elemen terurut dari outputArray ke inputArray
    for (int i = 0; i < size; i++) {
        array[i] = outputArray[i];
    }
}
```

array	2	5	3	0	2	3	0	3

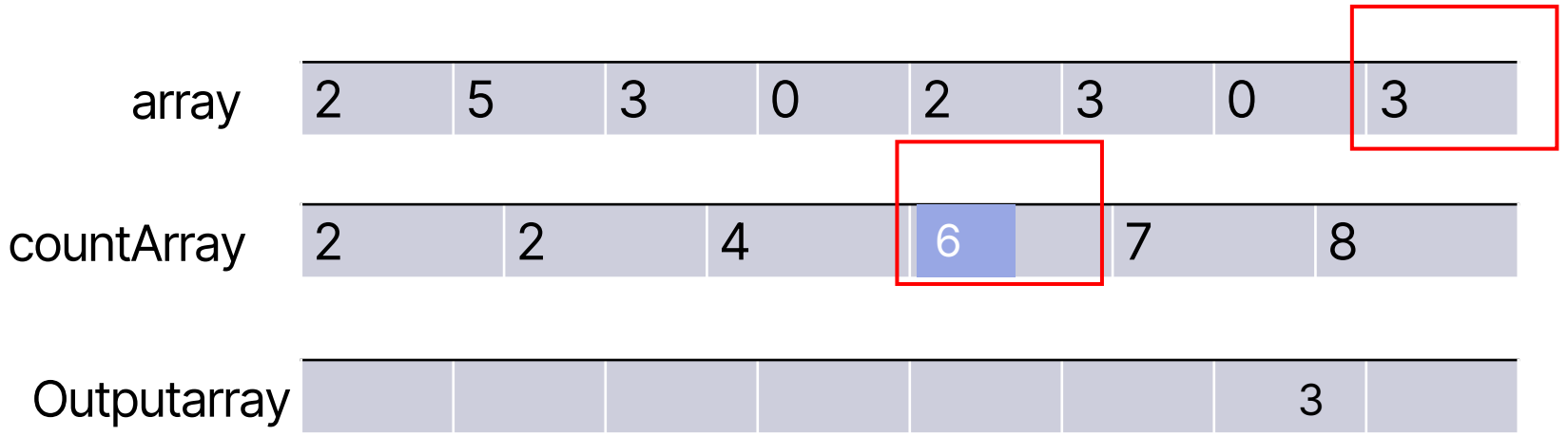
countArray	2	2	4	7	7	8	

Outputarray								

```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Pertama (i = 7, array[7] = 3):

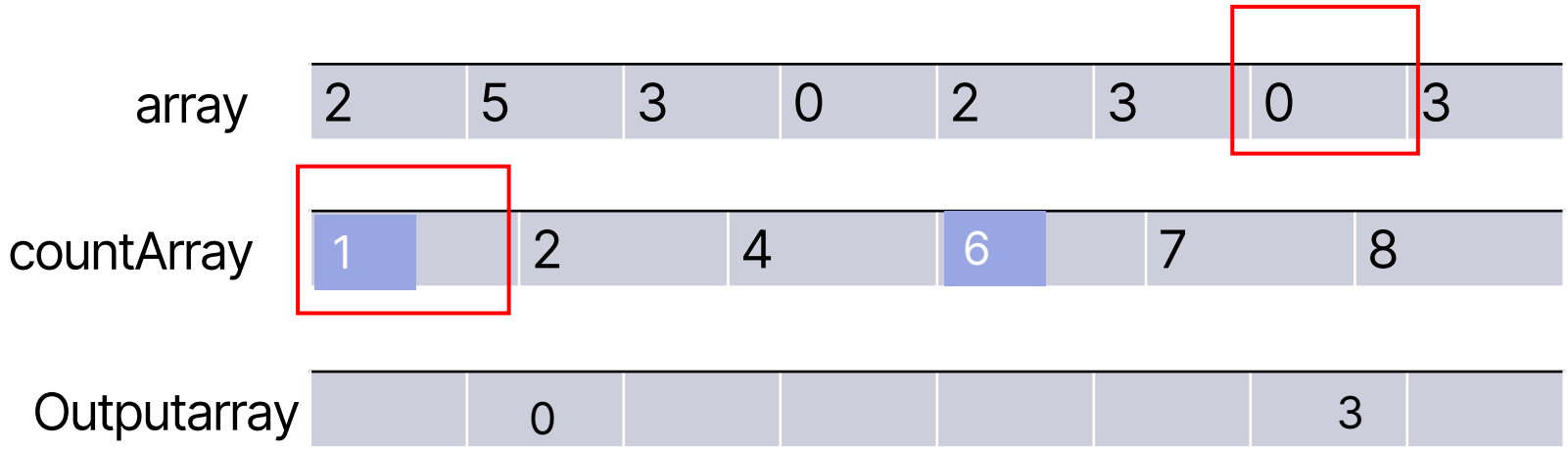
- Cari posisi elemen `3` di **countArray** (pada indeks 3), nilainya adalah `7`.
- Tempatkan elemen `3` di **outputArray** pada indeks `6` (`countArray[3] - 1 = 6`).
- Kurangi nilai **countArray** untuk elemen `3` (pada indeks 3) menjadi `6`.
- **outputArray** sekarang: `[ , , , , , , 3, ]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Kedua (i = 6, array[6] = 0):

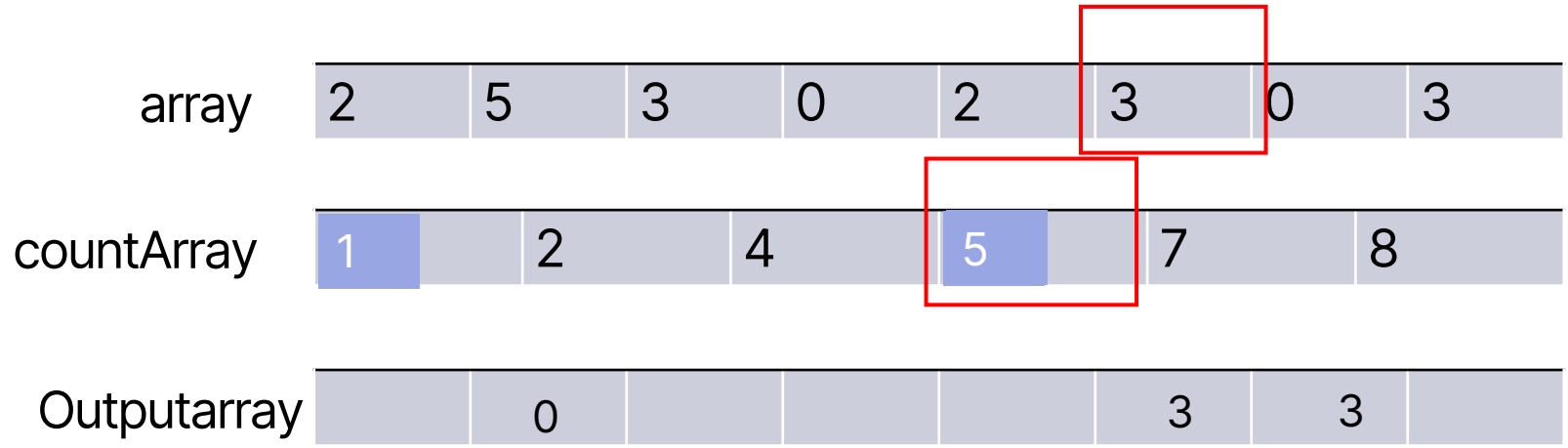
- Cari posisi elemen `0` di **countArray** (pada indeks 0), nilainya adalah `2`.
- Tempatkan elemen `0` di **outputArray** pada indeks `1` (`countArray[0] - 1 = 1`).
- Kurangi nilai **countArray** untuk elemen `0` menjadi `1`.
- **outputArray** sekarang: `[ , 0, , , , , 3, ]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Ketiga (i = 5, array[5] = 3):

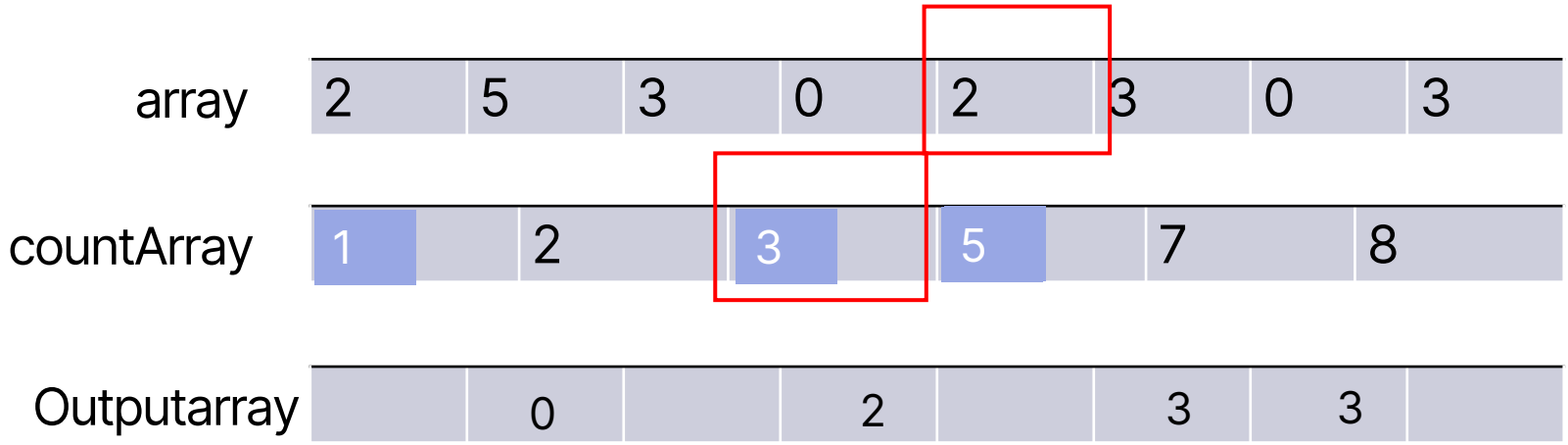
- Cari posisi elemen `3` di **countArray** (pada indeks 3), nilainya adalah `6`.
- Tempatkan elemen `3` di **outputArray** pada indeks `5` (`countArray[3] - 1 = 5`).
- Kurangi nilai **countArray** untuk elemen `3` menjadi `5`.
- **outputArray** sekarang: `[ , 0, , , , 3, 3, ]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Keempat (i = 4, array[4] = 2):

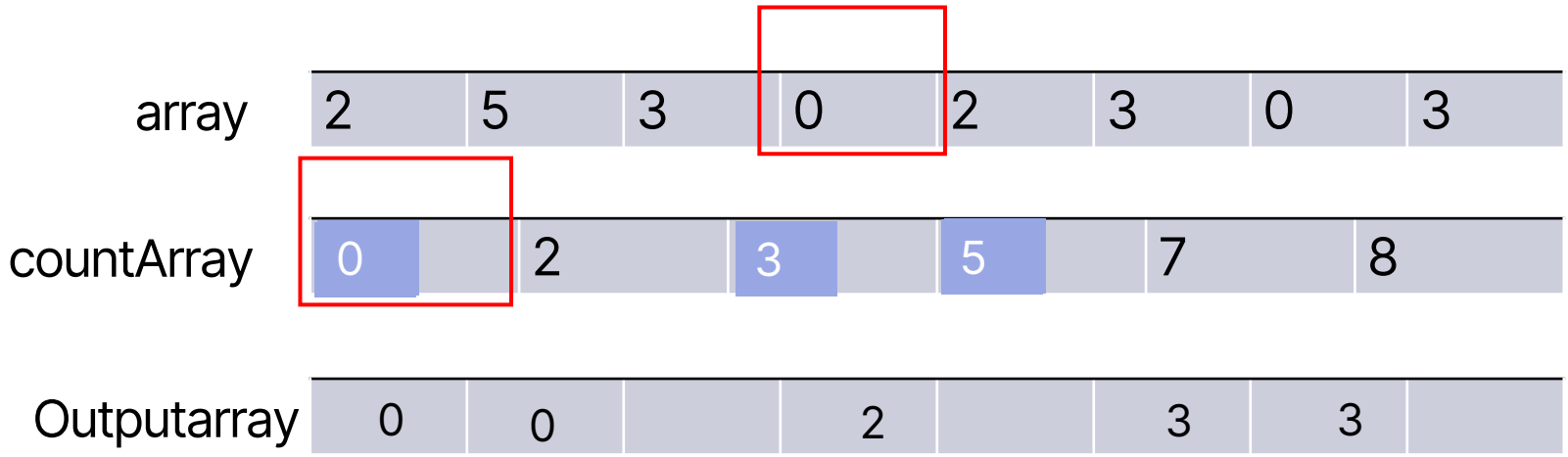
- Cari posisi elemen `2` di **countArray** (pada indeks 2), nilainya adalah `4`.
- Tempatkan elemen `2` di **outputArray** pada indeks `3` (`countArray[2] - 1 = 3`).
- Kurangi nilai **countArray** untuk elemen `2` menjadi `3`.
- **outputArray** sekarang: `[ , 0, , 2, , 3, 3, ]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Kelima (i = 3, array[3] = 0):

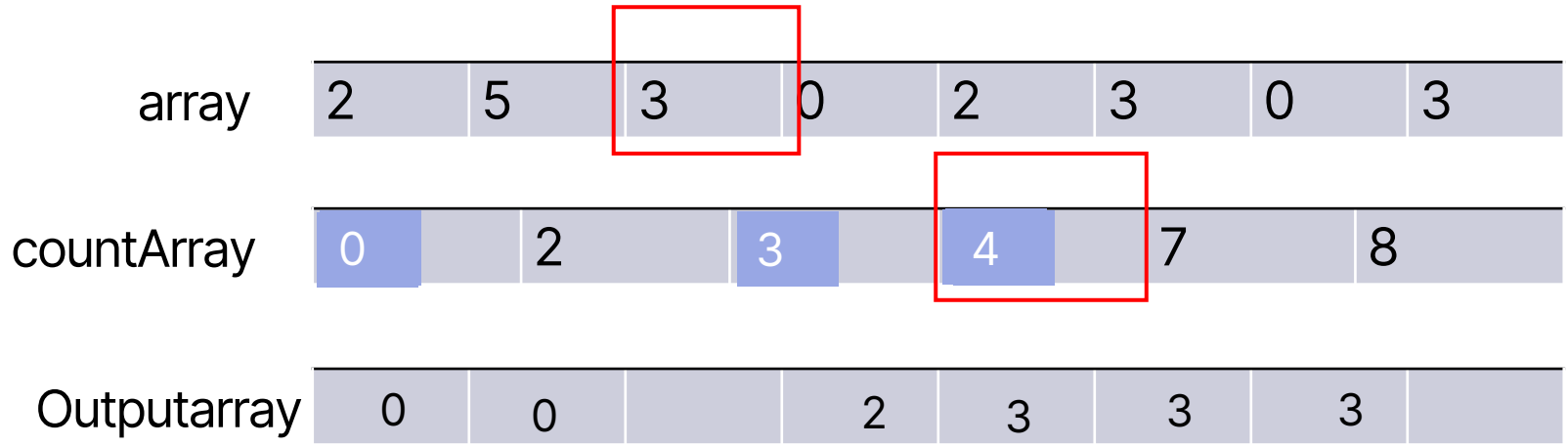
- Cari posisi elemen `0` di **countArray** (pada indeks 0), nilainya adalah `1`.
- Tempatkan elemen `0` di **outputArray** pada indeks `0` (`countArray[0] - 1 = 0`).
- Kurangi nilai **countArray** untuk elemen `0` menjadi `0`.
- **outputArray** sekarang: `[0, 0, , 2, , 3, 3, ]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Keenam (i = 2, array[2] = 3):

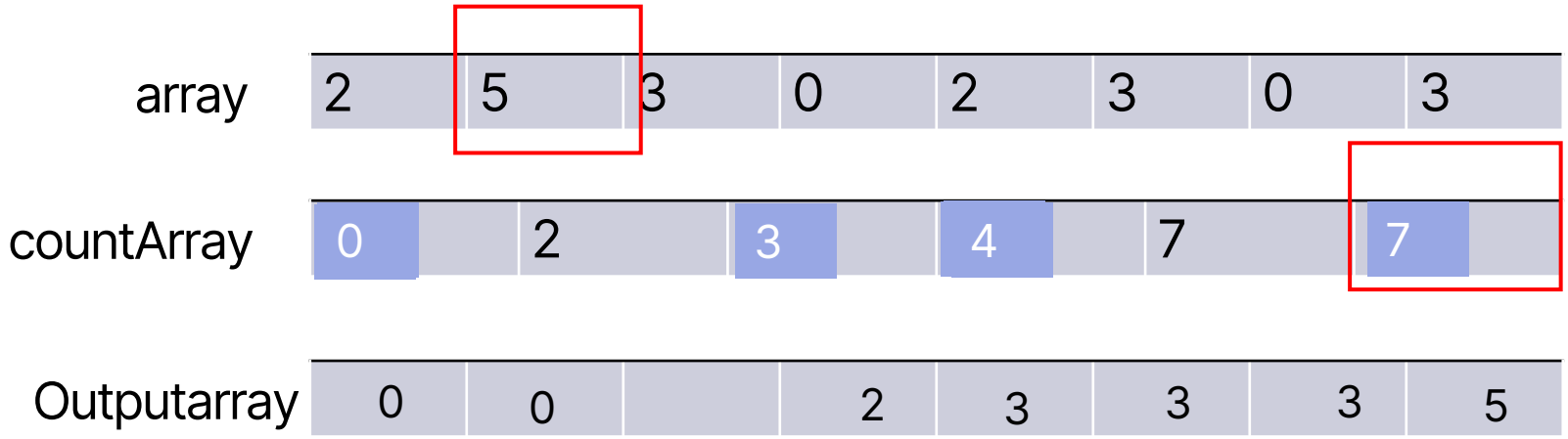
- Cari posisi elemen `3` di **countArray** (pada indeks 3), nilainya adalah `5`.
- Tempatkan elemen `3` di **outputArray** pada indeks `4` (`countArray[3] - 1 = 4`).
- Kurangi nilai **countArray** untuk elemen `3` menjadi `4`.
- **outputArray** sekarang: `[0, 0, , 2, 3, 3, 3, ]`.



```
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Ketujuh (i = 1, array[1] = 5):

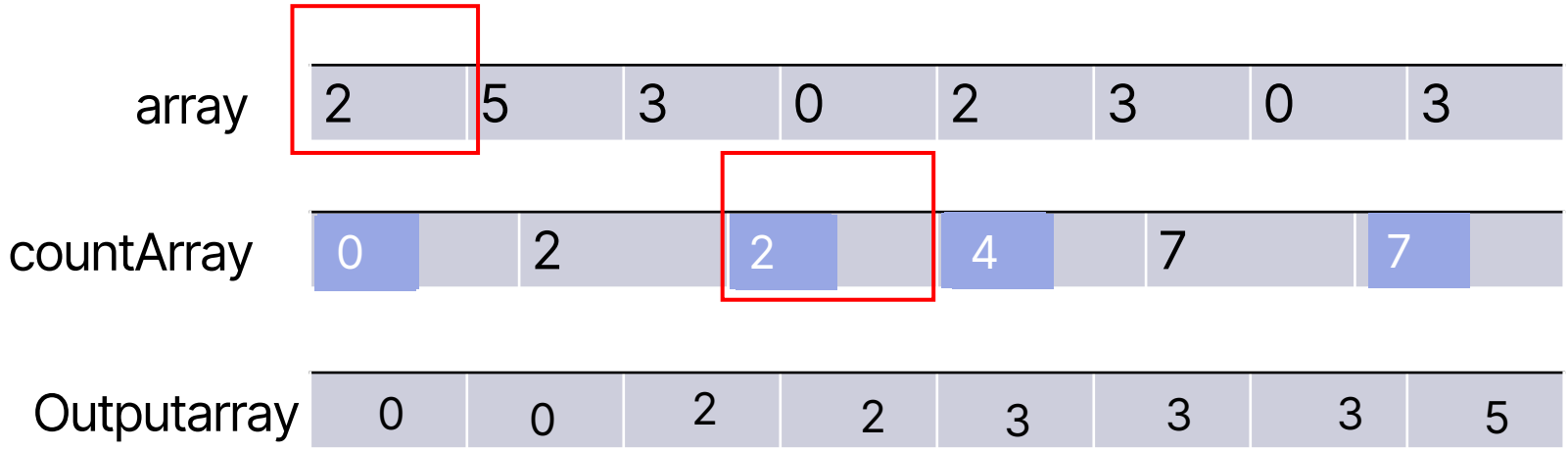
- Cari posisi elemen `5` di **countArray** (pada indeks 5), nilainya adalah `8`.
- Tempatkan elemen `5` di **outputArray** pada indeks `7` (`countArray[5] - 1 = 7`).
- Kurangi nilai **countArray** untuk elemen `5` menjadi `7`.
- **outputArray** sekarang: `[0, 0, , 2, 3, 3, 3, 5]`.



```java
// Isi outputArray dengan elemen-elemen yang terurut
for (int i = size - 1; i >= 0; i--) {
    outputArray[countArray[array[i]] - 1] = array[i];
    countArray[array[i]]--;  // Kurangi count setelah elemen ditempatkan
}
```

Iterasi Kedelapan (i = 0, array[0] = 2):

- Cari posisi elemen `2` di **countArray** (pada indeks 2), nilainya adalah `3`.
- Tempatkan elemen `2` di **outputArray** pada indeks `2` (`countArray[2] - 1 = 2`).
- Kurangi nilai **countArray** untuk elemen `2` menjadi `2`.
- **outputArray** sekarang: `[0, 0, 2, 2, 3, 3, 3, 5]`.



Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

IPB University
Bogor Indonesia

Counting Sort

- Jika data itu bersifat **diskret** dan kemungkinan nilainya **lebih kecil dari** n, maka akan lebih mudah untuk melakukan sort dengan menghitung terlebih dahulu frekuensi setiap data
- Prinsip ini dipakai didalam Counting Sort: hitung frekuensi setiap data, lalu enumerasi mulai dari nilai terkecil

```
1  COUNTING-SORT(A[1..n], k):
2      // 1) Hitung frekuensi setiap nilai
3      C ← new array[0..k] filled with 0
4      for i ← 1 to n:
5          C[A[i]] ← C[A[i]] + 1
6
7      // 2) Prefix-sum: C[v] = banyak elemen ≤ v
8      for v ← 1 to k:
9          C[v] ← C[v] + C[v-1]
10
11     B ← new array[1..n]
12     for i ← n downto 1:
13         v ← A[i]
14         pos ← C[v]
15         B[pos] ← A[i]
16         C[v] ← C[v] - 1
17
18     return B
```

- Counting Sort

IPB University
Bogor Indonesia

Counting Sort

- Berapa kompleksitas Counting Sort?
- Apakah dapat diterapkan pada data yang tidak bulat (pecahan/desimal)?

Counting Sort bekerja dengan:
1.Membuat array count dengan ukuran k (rentang nilai input).
2.Menghitung frekuensi setiap nilai → **O(n)**.
3.Membuat prefix sum → **O(k)**.
4.Mengisi array output → **O(n)**.
Sehingga total kompleksitas:

$$O(n + k)$$

•Jika **rentang nilai k ≈ n** → hampir linear.
•Tapi kalau k jauh lebih besar dari n → jadi **tidak efisien** (boros memori dan waktu).

IPB University
Bogor Indonesia

Kriteria pemilihan

- **Stabilitas**: apabila dua elemen memiliki **key** yang sama, urutannya harus tetap sebelum dan sesudah **sorting**
  - **Contoh:** Insertion, Merge, Counting
  - **Tidak stabil:** Quick
- **In-place:** tidak membutuhkan ruang (memori) tambahan
  - **Contoh:** Quick, Insertion
  - **Tidak in-place:** Merge
- Best/Worst/Average Case Complexity: O(n), O(n log n), O(n^2)

IPB University
Bogor Indonesia

Stabilitas

- ✅**Insertion Sort**
- Saat menyisipkan elemen, ia **tidak menukar posisi jika key sama**.
- Jadi elemen dengan key sama tetap dalam urutan aslinya.
-

- ✅ **Merge Sort**
- Saat merge, jika ada elemen dengan key sama di dua subarray, **diambil dulu dari array kiri**.
- Sehingga elemen yang muncul lebih dulu tetap dipertahankan.
-

- ✅ **Counting Sort**
- Prefix sum + **pengisian dari kanan ke kiri** menjamin bahwa elemen dengan key sama dimasukkan **dengan urutan asli**.
- Inilah kenapa Counting Sort *secara default dibuat stabil* (supaya bisa dipakai di Radix Sort).

IPB University
Bogor Indonesia

Praktik Analisis Algoritme di Industri

- **Skalabilitas dan Biaya**: kompleksitas mempengaruhi kemampuan dan system dan biayanya ketika jumlah input/user naik
- SLA/SLO: Harus mempertahankan **worst-case** supaya tetap dapat diterima (**managable**)
- Untuk ukuran data kecil, O(n^2) bisa jadi **lebih baik** dari O(n log n)
- Memori (kompleksitas ruang) juga penting, jika terlalu besar, maka system tidak akan dapat berjalan (crash)

IPB University
Bogor Indonesia

Latihan

```
1  QUICKSORT-LOMUTO(A, ℓ, r):
2      if ℓ ≥ r: return
3      p ← PARTITION-LOMUTO(A, ℓ, r)
4      QUICKSORT-LOMUTO(A, ℓ, p-1)
5      QUICKSORT-LOMUTO(A, p+1, r)
6
7  PARTITION-LOMUTO(A, ℓ, r):
8      pivot ← A[r]
9      i ← ℓ
10     for j ← ℓ to r-1:
11         if A[j] ≤ pivot:
12             swap A[i], A[j]
13             i ← i + 1
14     swap A[i], A[r]
15     return i
```

- Apa perbedaannya dengan algoritme Quick Sort standar?
- Bagaimana kompleksitasnya?

IPB University

Bogor Indonesia

Inspiring Innovation with Integrity

in Agriculture, Ocean and Biosciences for a Sustainable World